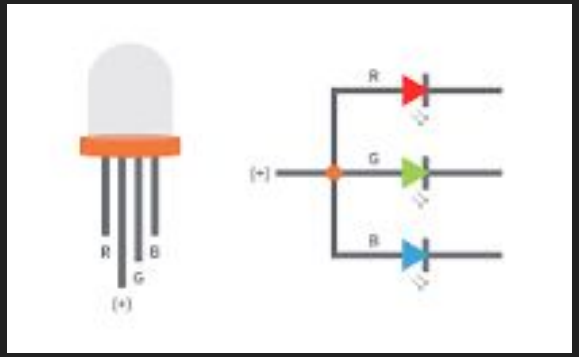
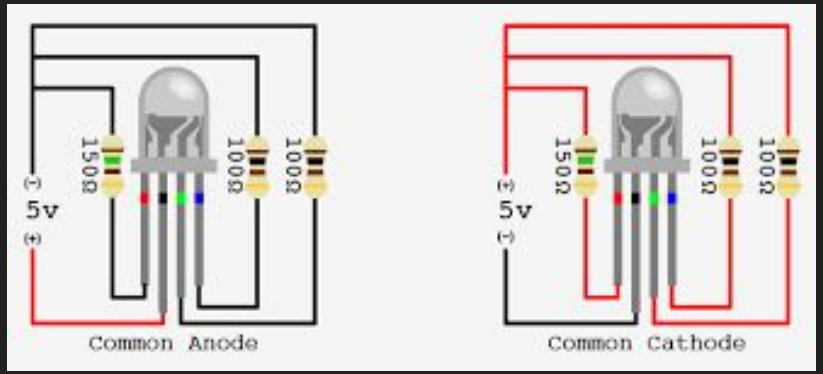
# Digital Electronics Interfacing

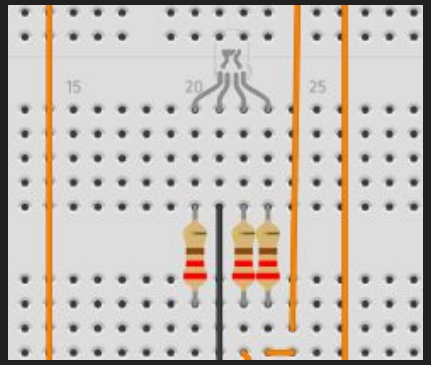
Lesson 9

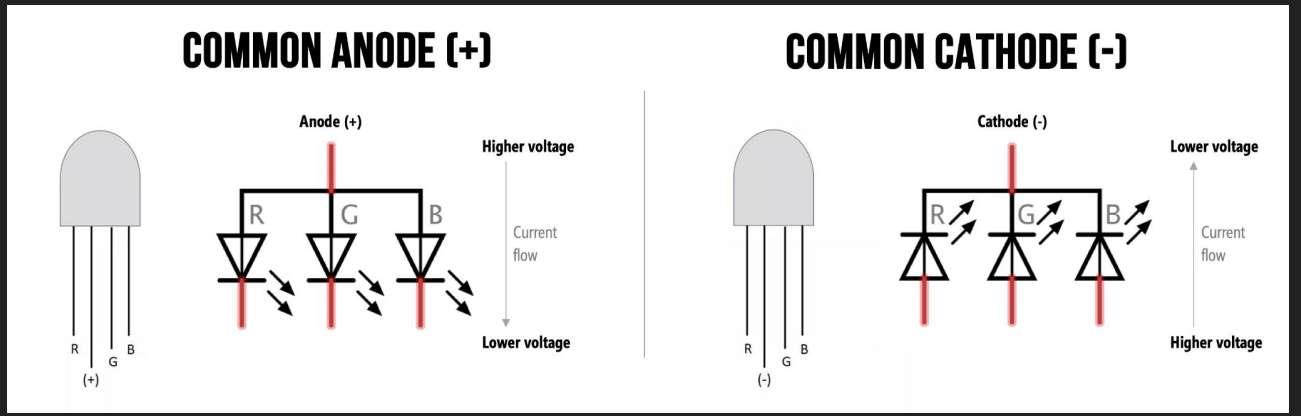
# Output Components-LED RGB



# Appearance

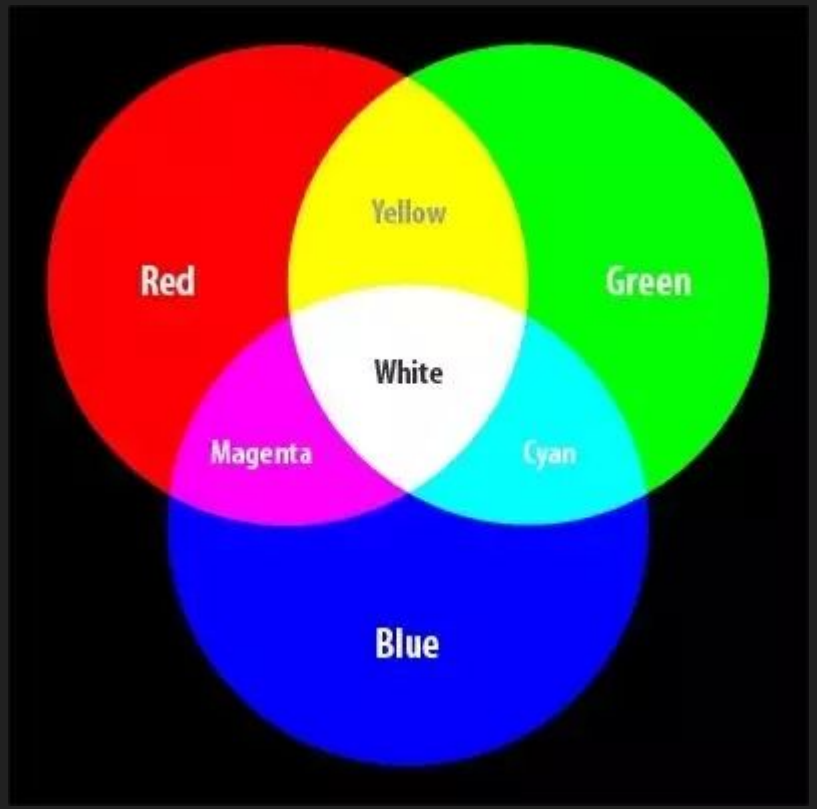
The PCB version of the RGB LED contains SMD resistors. As a result these do not require additional through hole resistors to be placed into the breadboard circuit. Most of these are common cathode and will need a connection to GND via the - labeled pin. R,G and B pins are also labeled.



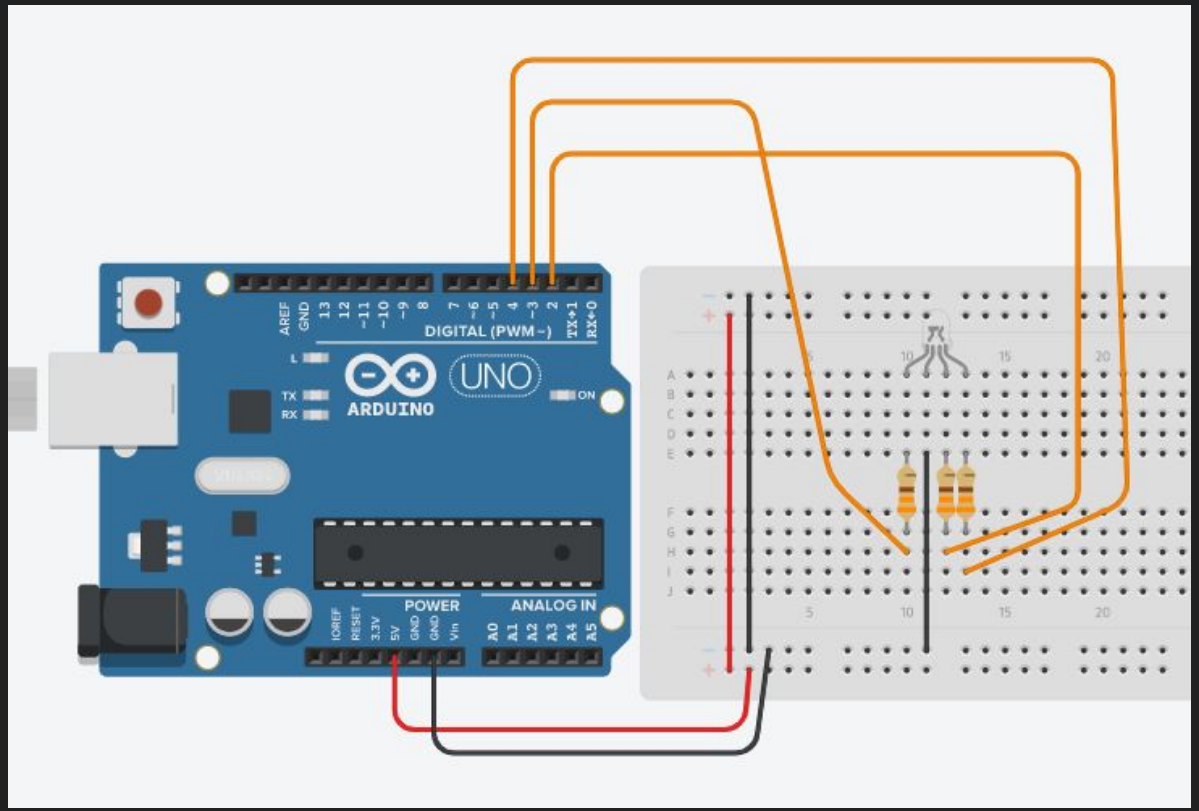
# Colour Combinations-LED RGB

You can display up to 7 different colours in the RGB by connecting the R,G and B pins according to the colour wheel.

For example to display Magenta we connect the R and B pins to HIGH and the G pin to LOW.



# LED RGB-Circuit and Code




```
// C++ code
//
void setup()
{
  pinMode(2, OUTPUT);
}

void loop()
{
  digitalWrite(2, HIGH);
  delay(1000); // Wait for 1000 millisecond(s)
  digitalWrite(2, LOW);
  delay(1000); // Wait for 1000 millisecond(s)

  digitalWrite(3, HIGH);
  delay(1000); // Wait for 1000 millisecond(s)
  digitalWrite(3, LOW);
  delay(1000); // Wait for 1000 millisecond(s)

  digitalWrite(4, HIGH);
  delay(1000); // Wait for 1000 millisecond(s)
  digitalWrite(4, LOW);
  delay(1000); // Wait for 1000 millisecond(s)

  digitalWrite(2, HIGH);
  digitalWrite(3, HIGH);
  digitalWrite(4, HIGH);
  delay(1000); // Wait for 1000 millisecond(s)
  digitalWrite(2, LOW);
  digitalWrite(3, LOW);
  digitalWrite(4, LOW);
  delay(1000); // Wait for 1000 millisecond(s)
}
```

# Data Types, Variables, Random Numbers and the Serial Monitor

```
long randNumber;

void setup() {
  Serial.begin(9600);

  // if analog input pin 0 is unconnected, random analog
  // noise will cause the call to randomSeed() to generate
  // different seed numbers each time the sketch runs.
  // randomSeed() will then shuffle the random function.
  randomSeed(analogRead(0));
}

void loop() {
  // print a random number from 0 to 299
  randNumber = random(300);
  Serial.println(randNumber);

  // print a random number from 10 to 19
  randNumber = random(10, 20);
  Serial.println(randNumber);

  delay(50);
}
```

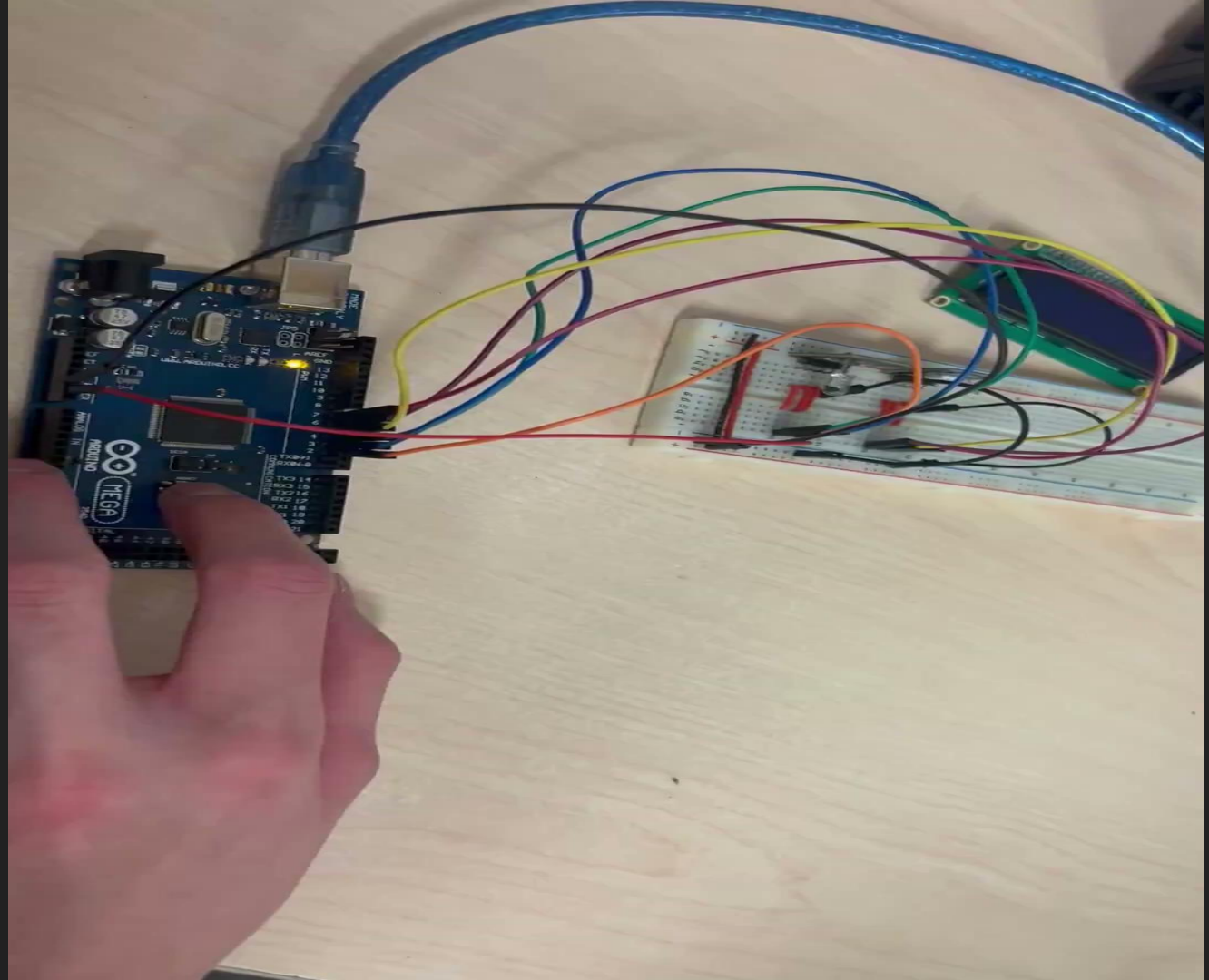
- **boolean** (8 bit) - simple logical true/false
- **byte** (8 bit) - unsigned number from 0-255
- **char** (8 bit) - signed number from -128 to 127. The compiler will attempt to interpret this data type as a character in some circumstances, which may yield unexpected results
- **unsigned char** (8 bit) - same as 'byte'; if this is what you're after, you should use 'byte' instead, for reasons of clarity
- **word** (16 bit) - unsigned number from 0-65535
- **unsigned int** (16 bit)- the same as 'word'. Use 'word' instead for clarity and brevity
- **int** (16 bit) - signed number from -32768 to 32767. This is most commonly what you see used for general purpose variables in Arduino example code provided with the IDE
- **unsigned long** (32 bit) - unsigned number from 0-4,294,967,295. The most common usage of this is to store the result of the `millis()` function, which returns the number of milliseconds the current code has been running
- **long** (32 bit) - signed number from -2,147,483,648 to 2,147,483,647
- **float** (32 bit) - signed number from -3.4028235E38 to 3.4028235E38. Floating point on the Arduino is not native; the compiler has to jump through hoops to make it work. If you can avoid it, you should. We'll touch on this later.

# Assignment (create the tinkercad prototype, show me and then build the actual circuit and show me again) /15 Tinkercad Prototype /15 Physical Circuit - App

Create a circuit with 2 RGB LEDs. Create a program that does the following in the order specified:

1. Turn on the red colour on each RGB at the same time.
2. Wait 1 second
3. Turn on the green colour on each RGB at the same time.
4. Wait 1 second.
5. Turn on the blue colour on each RGB at the same time
6. Wait 1 second
7. Show red, then yellow, then green, then cyan and so on through the rest of the colour wheel shown on slide #3. Use a half second delay in between colours and end with a white. Then continue and do the same for the second RGB
8. Turn both LEDs off then wait 2 seconds
9. Display a random colour on a random RGB. Do this 5 times with a 1 second delay in between.
10. Repeat steps 1 through 9 over and over again.

https://www.tinkercad.com/things/5b4EFIbvcZ8-glorious-bruticus


ARDUINO
MEGA